

VORTEX

High-Performance Layer 4 / Layer 7 Load Balancer

Project Type

Senior-level Go networking and systems engineering project.

Project Goal

Build a high-performance, production-inspired load balancer from scratch using Go.

VORTEX should support both:

- Layer 4 TCP load balancing
- Layer 7 HTTP load balancing

The project should focus heavily on:

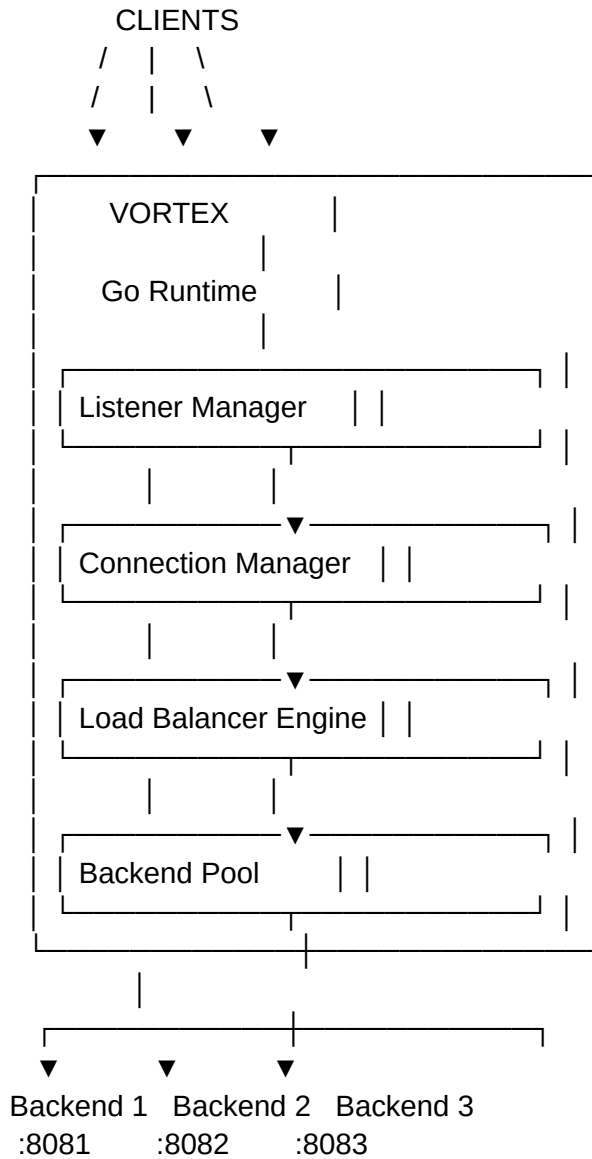
- network programming
- concurrency
- connection management
- scheduling algorithms
- health checking
- failure handling
- backpressure
- TLS termination
- observability
- runtime configuration
- performance benchmarking

The goal is not to recreate NGINX or HAProxy feature-for-feature.

The goal is to understand how a serious load balancer works internally.

1. Architecture

The final architecture should look approximately like this:



2. Core Technologies

Use:

Go 1.25+

Linux

TCP

HTTP/1.1

TLS

Prometheus

Docker
Docker Compose

Later:

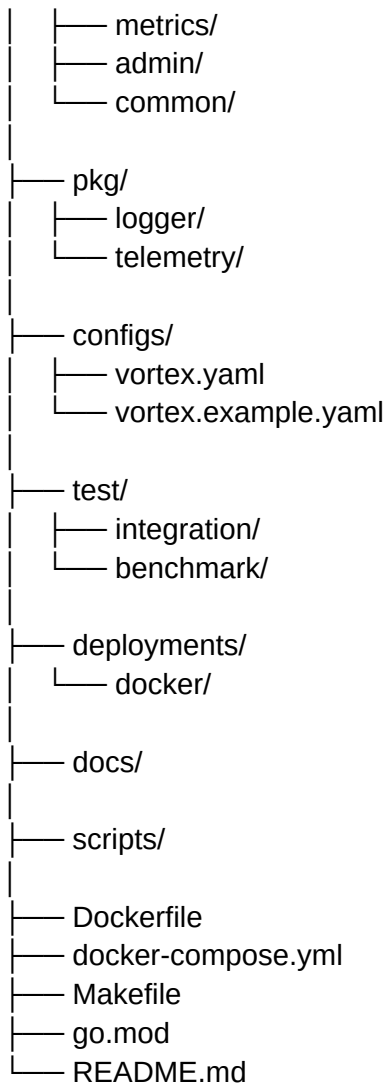
HTTP/2
WebSocket
eBPF
SO_REUSEPORT
kernel tuning

Do not add every advanced feature immediately.

3. Repository Structure

Use a clean Go project:

```
vortex/
├── cmd/
│   ├── vortex/
│   │   └── main.go
│   └── vortexctl/
│       └── main.go
├── internal/
│   ├── config/
│   ├── listener/
│   ├── connection/
│   ├── proxy/
│   ├── protocol/
│   ├── http/
│   ├── tcp/
│   ├── backend/
│   ├── balancer/
│   ├── health/
│   ├── tls/
│   └── reload/
```



Keep most implementation details under `internal`.

4. Configuration

VORTEX should use a configuration file.

Example:

server:

workers: 8

listeners:

- name: http
address: ":8080"
protocol: http

- name: tcp
address: ":9000"
protocol: tcp

backends:

- name: app-1
address: "10.0.0.11:8080"
weight: 1

- name: app-2
address: "10.0.0.12:8080"
weight: 2

load_balancing:

algorithm: weighted_round_robin

health_check:

enabled: true
interval: 5s
timeout: 2s
unhealthy_threshold: 3
healthy_threshold: 2

timeouts:

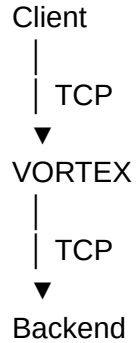
connect: 5s
read: 30s
write: 30s
idle: 60s

Environment variables should be able to override sensitive or deployment-specific values.

5. Layer 4 Load Balancing

Implement TCP proxying first.

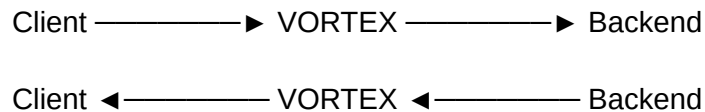
The flow:



VORTEX should:

1. accept the client connection
2. select a backend
3. establish a backend connection
4. proxy data in both directions
5. detect connection closure
6. clean up resources

Conceptually:



Use bidirectional streaming.

6. TCP Connection Handling

Create a connection abstraction.

It should track:

connection ID
client address
backend address
created time
bytes received

bytes sent
duration
state

States:

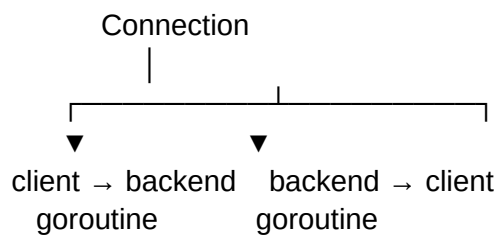
ACCEPTED
CONNECTING
ESTABLISHED
CLOSING
CLOSED

Never leak connections.

Every accepted connection must eventually be closed.

7. Goroutines

A basic TCP connection can initially use two goroutines:



However, do not stop there.

Later investigate:

- goroutine count
- memory overhead
- scheduler overhead
- connection scaling
- backpressure

Benchmark thousands of simultaneous connections.

8. Context and Cancellation

Every request/connection should have a lifecycle.

Use:

`context.Context`

for:

- shutdown
- timeouts
- cancellation
- health checks
- backend requests

When VORTEX shuts down, active operations should be cancelled gracefully.

9. Load-Balancing Algorithms

Implement a common interface:

```
type Balancer interface {  
    Next(ctx context.Context, pool *BackendPool) (*Backend, error)  
}
```

Implement:

Round Robin
Weighted Round Robin
Random
Least Connections
Least Response Time
IP Hash
Consistent Hashing
Power of Two Choices

10. Round Robin

Example:

Backend A
Backend B
Backend C

Request 1 → A
Request 2 → B
Request 3 → C
Request 4 → A
Request 5 → B

The implementation must be safe under concurrent access.

Do not introduce a global lock around every request if it can be avoided.

Investigate atomic counters.

11. Weighted Round Robin

Example:

Backend A → weight 5
Backend B → weight 2
Backend C → weight 1

Traffic should approximately follow:

A A A A A B B C

Do not assume exact distribution for small request counts.

Benchmark statistical distribution over large request volumes.

12. Least Connections

Each backend tracks active connections:

Backend A → 20

Backend B → 7

Backend C → 14

New connection:

→ Backend B

The counter must be concurrency-safe.

Investigate:

sync/atomic

instead of using a mutex everywhere.

13. Least Response Time

Track:

average latency

recent latency

active connections

Then select the backend with the best score.

Do not use an unlimited historical average.

Implement a moving window or exponentially weighted measurement.

14. Power of Two Choices

Instead of inspecting every backend:

choose two random backends
compare them
choose the better one

This is an interesting algorithm to benchmark against:

Round Robin
Least Connections
Power of Two Choices

Compare:

- throughput
 - latency
 - CPU
 - backend distribution
-

15. Backend Pool

Create:

BackendPool

Responsibilities:

add backend
remove backend
enable backend
disable backend
find healthy backend
track active connections
track latency
track failures

Backend state:

UP

DOWN

DRAINING

16. Backend Draining

When removing a backend, do not immediately terminate existing connections.

Instead:

UP

↓

DRAINING

↓

no new connections

↓

existing connections finish

↓

DOWN

This is essential for graceful infrastructure changes.

17. Health Checks

VORTEX must continuously check backend health.

For TCP:

connect()

For HTTP:

GET /health

Configuration:

health_check:

interval: 5s

timeout: 2s

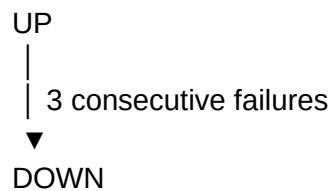
unhealthy_threshold: 3

healthy_threshold: 2

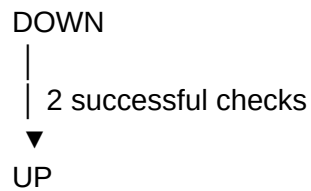
18. Health State Machine

Do not immediately mark a backend down after one failed request.

Example:



Recovery:



This prevents flapping.

19. Failure Handling

Simulate:

backend crash

backend timeout

connection refused
slow backend
network failure
partial response
connection reset

VORTEX should react predictably.

For example:

Backend A DOWN

↓

Scheduler excludes A

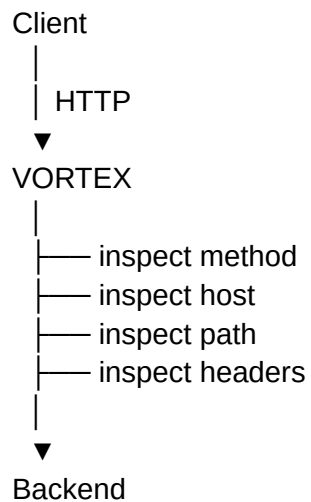
↓

Traffic goes to B/C

20. HTTP Layer 7 Proxy

Once TCP works, implement HTTP.

Architecture:



21. HTTP Routing

Support rules such as:

routes:

- host: api.example.com
path: /api
backend_pool: api
- host: app.example.com
path: /
backend_pool: frontend

Requests:

/api/users

↓

API pool

/

/index.html

↓

Frontend pool

22. Header Manipulation

Support:

X-Forwarded-For
X-Forwarded-Proto
X-Forwarded-Host

Example:

Client

↓

VORTEX

↓

X-Forwarded-For: client-ip

↓

Backend

Be careful not to blindly trust client-provided forwarding headers.

23. TLS Termination

Support:

Client

| HTTPS

▼

VORTEX

| HTTP

▼

Backend

VORTEX terminates TLS.

Configuration:

tls:

enabled: true

certificate: /etc/vortex/server.crt

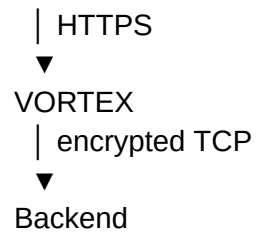
key: /etc/vortex/server.key

Later support SNI and multiple certificates.

24. TLS Passthrough

Eventually implement:

Client



VORTEX does not decrypt the connection.

This is an L4 mode.

25. Keep-Alive

HTTP keep-alive must be supported.

Without keep-alive:

request
connect
request
close

request
connect
request
close

With keep-alive:

connect
|— request
|— request
|— request
|— request
close

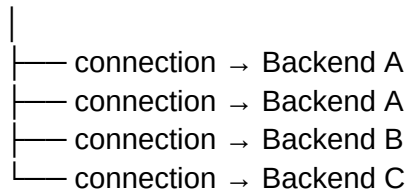
Benchmark the difference.

26. Connection Pooling

For HTTP proxying, investigate backend connection reuse.

Concept:

VORTEX



Reuse connections where safe.

Configure:

max idle connections
max connections per backend
idle timeout
connection timeout

27. Timeouts

Every network operation must have a timeout.

Implement:

connect timeout
read timeout
write timeout
idle timeout
request timeout
health-check timeout
shutdown timeout

Never rely on infinite network operations.

28. Backpressure

This is a critical systems concept.

Imagine:

Client

↓

VORTEX

↓

Slow Backend

If the backend can process only:

100 req/s

but clients send:

10000 req/s

VORTEX must not consume unlimited memory.

Implement:

bounded queues

connection limits

timeouts

request limits

Study what happens under overload.

29. Rate Limiting

Add optional rate limiting.

Support:

per-IP

per-route
per-host
global

Example:

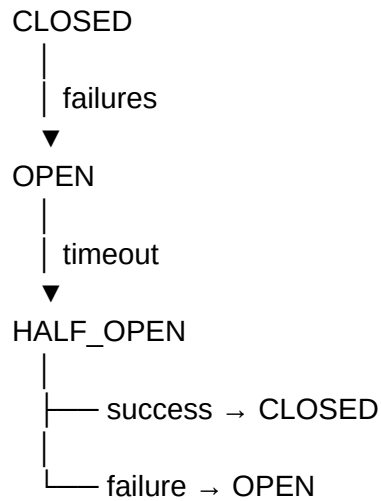
rate_limit:
 requests_per_second: 1000
 burst: 2000

Initially use an in-memory token bucket.

30. Circuit Breaker

Implement backend circuit breaking.

States:



This prevents repeatedly sending traffic to a failing backend.

31. Retry Policy

Be careful with retries.

Do NOT blindly retry every request.

For example:

GET → potentially retry

POST → potentially unsafe

Only retry operations where semantics allow it.

Support:

connection failure

timeout

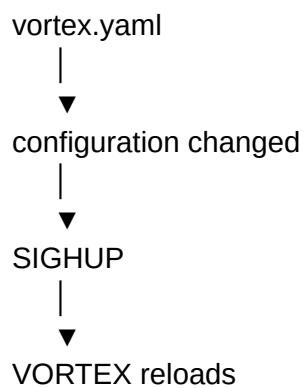
5xx

only under explicit configuration.

32. Graceful Reload

VORTEX should support configuration reload without stopping the process.

Example:



Existing connections should continue.

New configuration applies to new traffic.

33. Admin API

Create an internal admin API.

Example:

```
GET /admin/backends
GET /admin/backends/{id}
POST /admin/backends/{id}/enable
POST /admin/backends/{id}/disable
POST /admin/backends/{id}/drain
GET /admin/stats
GET /admin/config
```

Protect the admin API.

It should never be exposed publicly by default.

34. Runtime Statistics

Expose:

```
active connections
total connections
requests
responses
errors
bytes received
bytes sent
backend health
backend latency
```

Example:

```
{
  "connections": {
```

```
"active": 1240,  
"total": 9238472  
},  
"requests": {  
  "total": 12493821,  
  "errors": 182  
}  
}
```

35. Prometheus Metrics

Expose `/metrics`.

Examples:

```
vortex_connections_active  
vortex_connections_total
```

```
vortex_requests_total  
vortex_request_duration_seconds
```

```
vortex_backend_connections  
vortex_backend_requests_total  
vortex_backend_errors_total
```

```
vortex_backend_health  
vortex_backend_latency_seconds
```

```
vortex_bytes_received_total  
vortex_bytes_sent_total
```

Add labels carefully.

Do not create unbounded cardinality.

Never use things like arbitrary URLs or user IDs as metric labels.

36. Logging

Use structured logging.

Example:

```
{  
  "level": "info",  
  "event": "backend_selected",  
  "backend": "app-2",  
  "algorithm": "least_connections",  
  "connection_id": "abc123"  
}
```

Log important events:

listener_started
connection_accepted
backend_selected
backend_down
backend_up
configuration_reloaded
connection_closed
tls_error
proxy_error

Do not log sensitive data.

37. Distributed Tracing

For HTTP requests, optionally integrate OpenTelemetry.

Trace:

Client
↓
VORTEX
↓
Backend

Record:

routing decision
backend selected
connection latency
response latency
errors

38. Admin CLI

Create:

vortexctl

Commands:

vortexctl status

vortexctl backend list

vortexctl backend show app-1

vortexctl backend disable app-1

vortexctl backend enable app-1

vortexctl backend drain app-1

vortexctl stats

vortexctl reload

The CLI communicates with the admin API.

39. Security

Implement:

TLS

secure admin API

request size limits

connection limits

timeout enforcement

header sanitization

rate limiting

authentication for admin operations

Later:

mTLS

40. Resource Limits

VORTEX must have configurable limits.

Examples:

limits:

max_connections: 100000

max_connections_per_backend: 10000

max_request_body_mb: 10

max_header_size_kb: 64

When limits are reached, fail gracefully.

41. Graceful Shutdown

On:

SIGTERM

SIGINT

perform:

stop accepting connections

↓

stop routing new requests

↓

drain existing connections

↓

wait for timeout

↓

close remaining connections

↓

flush metrics/logs

↓

exit

Example:

Shutdown timeout = 30 seconds

42. Testing

Testing is a major part of VORTEX.

Unit Tests

Test:

Round Robin

Weighted Round Robin

Least Connections

IP Hash

health state machine

circuit breaker

rate limiter

configuration parser

routing

43. Integration Tests

Create several fake backends.

For example:

backend-1 :8081

backend-2 :8082

backend-3 :8083

Send requests through VORTEX.

Verify:

requests are distributed

failed backends are removed

recovered backends return

draining works

timeouts work

44. Failure Tests

Automate:

kill backend

restart backend

make backend slow

drop connection

return 500

return malformed response

delay response

Then verify VORTEX behavior.

45. Concurrency Tests

Test:

10,000 concurrent clients

and verify:

no race conditions

no connection leaks

no deadlocks

no negative counters

no backend overload beyond limits

Always run:

go test -race ./...

46. Load Testing

Use an external load generator.

Examples:

wrk

hey

k6

vegeta

Test:

10k requests

100k requests

1M requests

Measure:

requests/sec

p50
p95
p99
p99.9
CPU
memory
connections

47. Benchmark the Algorithms

Create controlled benchmarks.

Example:

Backends: 10
Requests: 1,000,000
Concurrency: 1000

Compare:

Round Robin
Least Connections
Power of Two Choices
Weighted Round Robin

Record:

throughput
latency
CPU
allocations
backend distribution

48. Go Profiling

Use:

pprof

Investigate:

CPU
heap
goroutines
mutex contention
blocking

Look for:

unnecessary allocations
lock contention
excessive goroutines
copying
buffer allocation

Do not optimize based purely on intuition.

Measure first.

49. Memory Optimization

Investigate buffer reuse.

For example:

`sync.Pool`

can potentially be used for temporary buffers.

But benchmark before introducing it.

The goal is not to use `sync.Pool` everywhere.

The goal is:

less allocation

less GC pressure
higher throughput

50. Advanced Linux Networking

Once the normal implementation is working, investigate:

epoll
SO_REUSEPORT
TCP_NODELAY
TCP keepalive
socket buffers
TCP backlog

The standard Go networking stack should be the starting point.

Do not immediately replace it with custom syscall code.

First understand what Go is already doing.

51. Optional Advanced Mode

After the normal implementation is complete, create:

VORTEX-X

as an experimental high-performance branch.

Investigate:

custom event loops
Linux epoll
io_uring
zero-copy techniques
splice()
sendfile()
SO_REUSEPORT

CPU affinity
NUMA
eBPF

This should be experimental.

Keep the normal VORTEX implementation clean.

52. Docker Environment

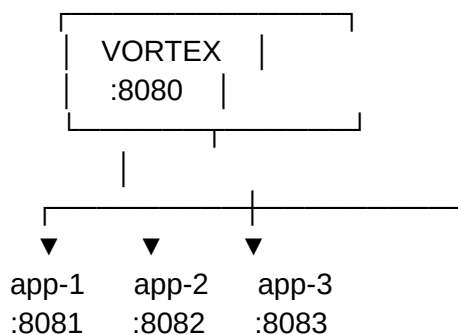
Create:

docker-compose.yml

with:

vortex
backend-1
backend-2
backend-3
prometheus
grafana

Example:



53. Demo Application

Create a tiny backend application for testing.

Each backend should return:

```
{  
  "server": "backend-1",  
  "timestamp": "...",  
  "request_id": "..."  
}
```

This makes load balancing immediately visible.

54. Final Demo

The final demonstration should look like:

1. Start VORTEX
2. Start backend-1
3. Start backend-2
4. Start backend-3
5. Send 1000 requests
6. Observe distribution
7. Stop backend-2
8. Health checker detects failure
9. VORTEX removes backend-2
10. Traffic continues through backend-1/3
11. Restart backend-2
12. Health checker detects recovery

13. backend-2 returns to the pool
 14. Put backend-1 into DRAINING
 15. New connections avoid backend-1
 16. Existing connections finish
 17. Reload configuration
 18. Change algorithm
 19. Benchmark all algorithms
 20. Run 10,000 concurrent connections
 21. Inspect Prometheus
 22. Inspect pprof
 23. Gracefully shut down VORTEX
-

55. Development Phases

Phase 1 — Go Foundation

Implement:

project structure
configuration
logging
HTTP server
graceful shutdown
health endpoint

Phase 2 — TCP Proxy

Implement:

TCP listener
connection handling
backend connection
bidirectional proxy
timeouts

At this point VORTEX should already function as a basic L4 load balancer.

Phase 3 — Backend Pool

Implement:

backend registry
backend states
connection counters
backend lifecycle

Phase 4 — Load Balancing Algorithms

Implement:

Round Robin
Weighted Round Robin
Least Connections
Random

Then:

Power of Two Choices
IP Hash
Consistent Hashing

Phase 5 — Health Checking

Implement:

- TCP checks
- HTTP checks
- failure thresholds
- recovery thresholds
- backend state transitions

Phase 6 — HTTP Proxy

Implement:

- HTTP routing
- Host routing
- Path routing
- header forwarding
- keep-alive
- timeouts

Phase 7 — TLS

Implement:

- TLS termination
- TLS passthrough
- SNI
- certificate management

Phase 8 — Resilience

Implement:

- circuit breaker
- rate limiting
- connection limits
- backpressure
- retry policy

Phase 9 — Runtime Management

Implement:

admin API
CLI
configuration reload
backend draining
runtime statistics

Phase 10 — Observability

Implement:

Prometheus
Grafana
structured logs
OpenTelemetry
pprof

Phase 11 — Performance

Benchmark:

throughput
latency
connections
memory
CPU
algorithms

Run:

go test -race ./...
go test -bench=. ./...

Use pprof to investigate bottlenecks.

Phase 12 — Linux Optimization

Investigate:

epoll
SO_REUSEPORT
TCP tuning
socket buffers
zero-copy
io_uring
eBPF

Only after profiling demonstrates a real bottleneck.

56. Definition of Done

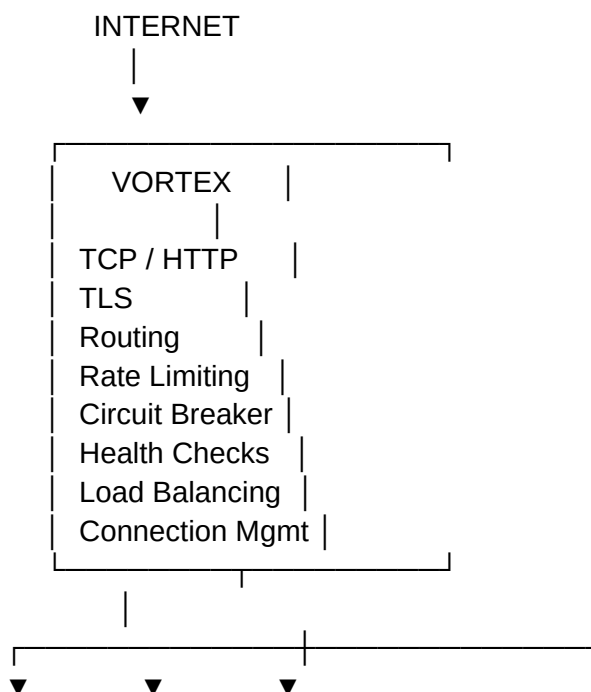
VORTEX is considered complete when it can:

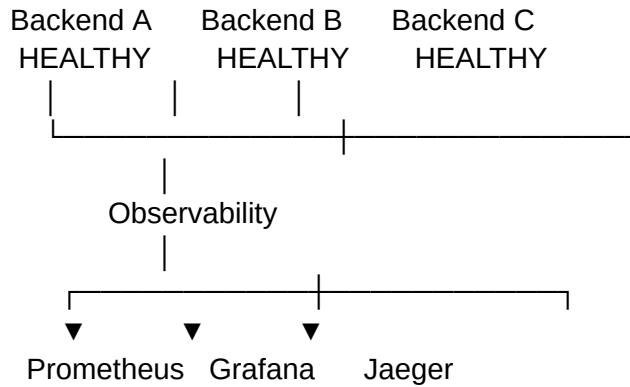
- accept TCP connections
- proxy TCP traffic
- load balance TCP connections
- perform HTTP reverse proxying
- perform HTTP routing
- support multiple backend pools
- support multiple load-balancing algorithms
- perform health checks
- detect backend failures
- detect backend recovery
- drain backends
- terminate TLS
- support TLS passthrough
- support HTTP keep-alive
- enforce connection limits
- implement backpressure

- implement rate limiting
 - implement circuit breaking
 - provide graceful shutdown
 - reload configuration
 - expose an admin API
 - provide a CLI
 - expose Prometheus metrics
 - support distributed tracing
 - provide structured logs
 - pass race detection
 - have integration tests
 - have failure tests
 - have concurrency tests
 - have load tests
 - have benchmarks
 - support profiling
 - run in Docker
 - document its architecture
-

57. Final Architecture

The final VORTEX architecture should look like:





58. What Makes VORTEX a Senior-Level Project

The important part is not the reverse proxy itself.

The interesting problems are:

How do you safely manage 100,000 connections?

How do you prevent connection leaks?

How do you avoid lock contention?

How do you detect unhealthy backends?

How do you prevent backend flapping?

How do you handle a slow backend?

How do you implement backpressure?

How do you gracefully remove a backend?

How do you reload configuration without dropping traffic?

How do you distribute traffic fairly?

How do you measure latency correctly?

How do you minimize allocations?

How does the Go scheduler affect performance?

Where is the bottleneck?

What happens when the system is overloaded?

Those are the questions VORTEX should force you to answer.

59. Ultimate Challenge

After completing everything above, run a final benchmark:

VORTEX

- |
- |— 10 backend servers
- |
- |— 100,000 concurrent connections
- |
- |— 1,000,000+ requests
- |
- |— 1 Gbit/s+ traffic

Measure:

RPS

p50 latency

p95 latency

p99 latency

p99.9 latency

CPU utilization

memory usage

goroutine count

GC activity

network throughput

backend distribution

Then profile VORTEX and identify its top three bottlenecks.

Optimize them.

Run the benchmark again.

The result should be a measurable before/after performance comparison.

60. Project Philosophy

Build VORTEX in this order:

Simple

↓

Correct

↓

Concurrent

↓

Resilient

↓

Observable

↓

Benchmarkable

↓

Optimized

Do not start by trying to make it the fastest load balancer in the world.

First make it correct.

Then make it concurrent.

Then make it resilient.

Then measure it.

Only then optimize it.

That development process is itself an important part of the project.